

SALESFORCE VALIDATION RULE MANAGER

Full-Stack Developer Interview & Code Walkthrough

About This Project Guide:

This guide compiles the complete technical documentation for the Salesforce Validation Rule Manager. It details the system architecture, authentication sequence using OAuth 2.0 with PKCE, XML metadata compilation, ZIP deployments to the Salesforce Metadata REST API, and a line-by-line analysis of all 19 frontend and backend code files. Use it to prepare for the technical review and live coding portion of the interview.

Architecture & Stack Summary:

- Frontend: React, Vite, React Router, custom Vanilla CSS
- Backend: Node.js, Express, Express Sessions, Axios REST clients
- Salesforce Integration: Tooling API (SOQL) & Metadata API (ZIP deployments)
- Security: OAuth2 authorization flow with PKCE code verifier
- Live Deployment: Split domains (Vercel Frontend / Render Backend with secure proxy trust)

Part 1: System Design & Interview Preparation

Salesforce Validation Rule Manager -- Complete Interview Preparation

This guide is designed to prepare you to walk through this project in technical interviews. It explains the project's architecture, details every file and its location, explains advanced concepts (such as PKCE, cookies, and Metadata deployments), and lists the top 10 interview questions you are likely to be asked.

> [!TIP]

> To read this file in VS Code in a formatted preview layout, open this file and press **Ctrl + Shift + V**.

> To save this as a PDF, install the **"Markdown PDF"** extension in VS Code, right-click inside this file, and select **"Markdown PDF: Export (pdf)"**.

Part 1: How to Pitch the Project (The 1-Minute Elevator Pitch)

"For my project, I built a full-stack **Salesforce Validation Rule Manager** application. The goal of the app is to allow Salesforce administrators to view and toggle (enable/disable) Account validation rules instantly in a custom UI, without having to navigate the slow Salesforce Setup menus.

It uses **React and Vite** on the frontend, and a **Node.js + Express** server on the backend.

Technically, it connects to any Salesforce Developer Org or Sandbox dynamically using a secure **OAuth 2.0 Web Server flow with PKCE**. It queries rules using the **Tooling API**, packages metadata changes in memory using a **ZIP stream pipeline**, and deploys them back to Salesforce via the **Metadata API**. I also configured **cross-domain secure cookies** and reverse proxy trust to deploy the frontend on Vercel and the backend on Render."

Part 2: High-Level Architecture & Data Flow

sequenceDiagram

participant User as Browser / Frontend (Vercel)

participant Server as Express Backend (Render)

participant SFAuth as Salesforce OAuth Services

participant SFAPI as Salesforce APIs (Tooling & Metadata)

%% Authentication Flow

User->>Server: 1. Click Login (GET /api/auth/login?env=sandbox)

Note over Server: Generates state & PKCE verifier/challenge.
Saves verifier to session.

Server-->>User: 302 Redirect to Salesforce login url

User->>SFAuth: Log in & click "Allow Access"

```

    SFAuth-->>Server: 302 Redirect to Callback URL with auth code (GET
/api/auth/callback?code=...)
    Server->>SFAuth: Exchange Code + PKCE Verifier for Tokens (POST /oauth2/token)
    SFAuth-->>Server: Returns access_token, refresh_token, instance_url
    Note over Server: Saves tokens & user profile to session.<br/>Saves session to store.
    Server-->>User: 302 Redirect to Vercel /dashboard

%% Fetching Rules Flow
    User->>Server: 2. Load Dashboard (GET /api/rules)
    Server->>SFAPI: SOQL Query Account Validation Rules (Tooling API)
    SFAPI-->>Server: Returns raw Salesforce rule records
    Note over Server: Normalizes raw SF fields to clean JSON payload
    Server-->>User: Returns normalized rules list

%% Toggling Rules Flow
    User->>Server: 3. Toggle Rule (PUT /api/rules/:fullName/toggle)
    Server->>SFAPI: Fetch full rule details (formula, description, error message)
    SFAPI-->>Server: Returns validation rule detail
    Note over Server: Compiles package.xml & rule metadata XML.<br/>Zips XML documents
in-memory using archiver.
    Server->>SFAPI: Deploy ZIP package (POST Metadata API /deploy)
    SFAPI-->>Server: Returns deployId
    Server-->>User: Returns deployId
    Note over User: Polls status recursively (GET /api/rules/deploy-status/:id)
    User->>Server: GET /api/rules/deploy-status/:id
    Server->>SFAPI: Check status (GET Metadata API /deploy/:id)
    SFAPI-->>Server: Status = 'Succeeded'
    Server-->>User: Returns done=true, success=true
    Note over User: Refreshes list and displays success toast

```

Part 3: File-by-File Walkthrough & Code Explanations

Here is where every key file is located, what it does, and the technical details you should know.

1. Backend Configuration: [server/config/salesforce.js](file:///C:/Users/sreev/.gemini/antigravity/scratch)

* **What it does:** Holds Salesforce configuration variables read from environment variables and builds the initial OAuth authorization URL.

* **Key Code Explained:**

* `sanitizeEnvVar()` cleans up variables to strip out double quotes (`"`), single quotes (`'`), or backslashes (`\`) that might have been accidentally pasted in Render's dashboard.

* `buildAuthorizeUrl(state, codeChallenge, loginUrl)` dynamically accepts a `loginUrl` (`https://login.salesforce.com` for production/developer accounts or `https://test.salesforce.com` for sandboxes) to construct the correct login gateway.

2. Backend Entry Point: [server/server.js](file:///C:/Users/sreev/.gemini/antigravity/scratch/sf-rule-manage)

* **What it does:** Imports `dotenv/config` to load variables from a local `.env` file, imports the Express app, and listens on `process.env.PORT` (or defaults to `3001` locally).

3. Express Application Setup: [server/app.js](file:///C:/Users/sreev/.gemini/antigravity/scratch/sf-rule-manage)

* **What it does:** Sets up CORS, JSON parsers, session middleware, reverse proxy trust, routing, and global error handlers.

* **Key Code Explained:**

* `app.set('trust proxy', true)`: Essential for production. Tells Express that it is running behind reverse proxies (Render load balancers + Cloudflare) and should trust the `X-Forwarded-Proto` header to evaluate secure cookie protocols.

* `isProd`: Detects if the app is running in production by checking if `NODE_ENV === 'production'` or if the callback URL contains `onrender.com`.

* `cookie.sameSite: 'none'` and `cookie.secure: true`: Configures session cookies to support cross-domain requests. Since the frontend is on Vercel and the backend is on Render, cookies will be blocked unless SameSite is set to `none` and Secure is `true`.

4. Authentication Middleware: [server/middleware/authCheck.js](file:///C:/Users/sreev/.gemini/antigravity/scratch/sf-rule-manage)

* **What it does:** Protects all validation rule API endpoints. It checks if `req.session.sf` exists. If not, it blocks the request and returns a `401 Unauthorized` status, preventing unauthenticated clients from querying or modifying metadata.

5. Salesforce OAuth Service: [server/services/sfAuth.js](file:///C:/Users/sreev/.gemini/antigravity/scratch/sf-rule-manage)

* **What it does:** Communicates directly with Salesforce token, user info, and revocation endpoints.

* **Key Code Explained:**

* `exchangeCodeForTokens(code, codeVerifier, loginUrl)`: Sends a POST request to the token endpoint (`/services/oauth2/token`). It includes the `client_id`, `client_secret`, `code`, and the PKCE `code_verifier`.

* `refreshAccessToken(refreshToken, loginUrl)`: Uses the refresh token to request a new access token when it expires.

* `revokeToken(token, loginUrl)`: Calls the Salesforce revocation API to invalidate the access token when the user logs out.

6. Tooling API Service: [server/services/sfTooling.js](file:///C:/Users/sreev/.gemini/antigravity/scratch/sf-rule-manage)

* **What it does:** Interacts with Salesforce's Tooling API to fetch lists of validation rules.

* **Key Code Explained:**

* `getValidationRules(instanceUrl, accessToken)`: Runs a SOQL query against the `ValidationRule` object to fetch all validation rules on the `Account` object:

```
SELECT Id, ValidationName, Active, Description, ErrorMessage,
EntityDefinition.QualifiedApiName FROM ValidationRule WHERE EntityDefinition.QualifiedApiName
= 'Account'
```

* `getOrgName(instanceUrl, accessToken)`: Queries the `Organization` object to get the Salesforce company/org name for the dashboard header.

7. Metadata API Service: [server/services/sfMetadata.js](file:///C:/Users/sreevl.gemini/antigravity/scratch/)

* **What it does:** Handles compiling the XML files, packaging them into a ZIP archive in memory, and deploying the toggled rule back to Salesforce.

* **Key Code Explained:**

* **In-Memory Zipping:** Uses `archiver('zip')` to write the `package.xml` and rule XML files directly into a buffer stream, eliminating disk operations.

* **Deployment Call:** Sends a multipart form-data POST request containing the ZIP buffer to the Metadata API endpoint:

`/services/data/v59.0/metadata/deploy`

* `checkDeployStatus(instanceUrl, accessToken, deployId)`: Checks the progress of the deployment.

8. Authentication Router: [server/routes/auth.js](file:///C:/Users/sreevl.gemini/antigravity/scratch/sf-rule/)

* **What it does:** Express route handlers for `/login`, `/callback`, `/me`, and `/logout`.

* **Key Code Explained:**

* `/login`: Captures the `?env=sandbox` query from the frontend. Generates a random `codeVerifier` and its SHA256 `codeChallenge`. Saves `oauthState`, `codeVerifier`, and `loginUrl` to the session and calls `req.session.save()` before redirecting.

* `/callback`: Checks for CSRF `state` mismatch. Extracts `codeVerifier` and `loginUrl` from the session. Calls `exchangeCodeForTokens`, retrieves user info, and saves everything to `req.session.sf` and `req.session.user` before calling `req.session.save()` and redirecting to the dashboard.

9. Validation Rules Router: [server/routes/rules.js](file:///C:/Users/sreevl.gemini/antigravity/scratch/sf-rule/)

* **What it does:** Routes for fetching rules (`GET /`), toggling a rule (`PUT /:fullName/toggle`), and checking deploy status (`GET /deploy-status/:id`).

* **Key Code Explained:**

* `normalizeRule(record)`: Normalizes the Tooling API field keys to generic camelCase keys (`id`, `name`, `active`, `description`, `errorMessage`) so the frontend doesn't need to depend on Salesforce's exact field names.

* `/rules/:fullName/toggle`: Fetches the detailed XML contents of the rule via the Tooling API, flips the `Active` boolean flag, calls `deployRuleToggle()` to initiate the Metadata deploy, and returns the `deployId`.

10. Frontend API Wrapper: [client/src/services/api.js](file:///C:/Users/sreevl.gemini/antigravity/scratch/sf-rule/)

* **What it does:** Contains Axios requests and handles dynamic backend routing.

* **Key Code Explained:**

* `isLocal / apiBase`: Automatically checks `window.location.hostname`. If it is `localhost`, it routes requests to `http://localhost:3001`. If it is running live on Vercel, it routes them to your Render URL. This eliminates environment variable mismatch bugs.

* `login(env)`: Redirects the browser to `${base}/api/auth/login?env=${env}` to start the OAuth flow.

11. Custom Auth Hook: [client/src/hooks/useAuth.js](file:///C:/Users/sreevl.gemini/antigravity/scratch/sf-rule/)

* **What it does:** Coordinates login state across the frontend. It calls `/api/auth/me` on mount to check if the user has a valid cookie session. If so, it populates the `user` state; otherwise, it stops loading and exposes the `login(env)` and `logout()` functions.

12. Login Page Component: [client/src/pages/Login.jsx](file:///C:/Users/sreevl.gemini/antigravity/scrato)

* **What it does:** Renders the login UI card, a dropdown to select the environment (**production** vs **sandbox**), handles clicking the button, and reads **?error=** query parameters from the callback redirect to display errors.

13. Dashboard Page Component: [client/src/pages/Dashboard.jsx](file:///C:/Users/sreevl.gemini/antigra)

* **What it does:** Fetches the validation rules grid, manages the toggling states, recursively polls the deploy status of toggled rules, handles cleanup of timers on unmount, and displays toast success/error notifications.

Part 4: Technical Concepts You Must Understand Deeply

Interviewers will drill you on the security and architectural decisions of this app. Be prepared to explain these concepts:

1. SameSite Cookies & Split-Domain Sessions (Vercel vs. Render)

* **The Challenge:** The frontend (**sreevas-boorlas-projects.vercel.app**) and backend (**sf-rule-manager-backend.onrender.com**) are on different domains. Browsers block cookies on cross-domain requests by default (under **SameSite=Lax**).

* **The Solution:**

1. We configured the backend cookie to use **sameSite: 'none'** and **secure: true**.
2. We enabled **credentials: true** on the Axios client configuration to tell the browser to include session cookies in cross-origin API calls.
3. We enabled **app.set('trust proxy', true)** in Express. Since Render terminates HTTPS and forwards the request over HTTP, Express would ordinarily reject **secure: true** cookies thinking it's an insecure HTTP connection. Enabling **trust proxy** tells Express to trust the **X-Forwarded-Proto** header.

2. Session Write Race Condition (req.session.save)

* **The Challenge:** Express Session saves session data to memory asynchronously. In routes like **/login** and **/callback**, when you call **res.redirect()** immediately after writing to **req.session**, the browser redirects to the next page and sends a request before the backend has finished writing the session to memory. This causes session loss, resulting in the **State mismatch -- possible CSRF** error.

* **The Solution:** We call **req.session.save()** manually. The callback of this method only executes once the session has successfully written to memory. We perform the **res.redirect()** inside this callback, eliminating the race condition.

3. PKCE OAuth Flow (Proof Key for Code Exchange)

* **Why it was needed:** Modern Salesforce orgs enforce PKCE by default for security to prevent OAuth code interception attacks.

* **How we built it:**

1. During `/login`, the backend generates a random cryptographically secure string called the `codeVerifier`.
2. It hashes the verifier using `SHA256` and encodes it in a URL-safe base64 format to create the `codeChallenge`.
3. It stores the `codeVerifier` in the session and redirects the user to Salesforce with the `code_challenge` parameter.
4. When Salesforce redirects back with the `code` in the callback, the backend sends the raw `codeVerifier` along with the authorization code during the token exchange. Salesforce hashes the verifier and matches it against the challenge.

4. Metadata API Deployment vs. Tooling API Queries

* **The Difference:**

- * The **Tooling API** is designed for querying fine-grained metadata. We use it to run a simple, fast SOQL query to get the list of rules. We also use it to retrieve the metadata details of a single rule (e.g. its formula).
- * The **Metadata API** is required for deploying changes. You cannot update the `active` status of a validation rule using a simple HTTP PATCH. You must compile the rule back into its XML source metadata format, compress it into a ZIP file, and deploy it to the Metadata API endpoint `/services/data/v59.0/metadata/deploy`.

Part 5: Top 10 Mock Interview Questions & Answers

Q1: Why did you choose to build a Node.js/Express backend instead of making all Salesforce calls directly from the frontend?

> **Answer:** For security. Making Salesforce API calls directly from the frontend would require exposing the Connected App's `client_secret` to the browser, which is a major security vulnerability. By using an Express backend proxy, all sensitive client secrets and OAuth access/refresh tokens are stored securely in server-side sessions and never exposed to the client.

Q2: What is PKCE, and why did you implement it in this application?

> **Answer:** PKCE stands for Proof Key for Code Exchange. It is a security extension to the OAuth 2.0 authorization code flow. I implemented it because Salesforce orgs enforce PKCE to prevent authorization code interception attacks. By generating a dynamic `code_verifier` and sending a hashed `code_challenge` during redirect, we prove to Salesforce during token exchange that our backend was the one that initiated the login request.

Q3: Explain how you toggle a validation rule. Why is it not a simple REST update?

- > **Answer:** Salesforce does not allow updating validation rules via a simple PATCH request because validation rules are core metadata. To change its active state, we must perform a Metadata API deployment:
- > 1. Query the Tooling API to get the validation rule details (including its formula and description).
 - > 2. Compile an XML metadata document representing the rule, set the `active` flag to true/false, and

compile a `package.xml` manifest.

> 3. Zip both files in-memory using `archiver` and convert the ZIP to a base64 buffer.

> 4. Deploy the ZIP package to the Salesforce Metadata API deploy endpoint and poll the resulting deploy ID until it succeeds.

Q4: How did you handle the session cookies since Vercel and Render use different subdomains?

> **Answer:** Since the frontend (`vercel.app`) and backend (`onrender.com`) are on different domains, the browser treats session cookies as third-party. To allow cookie persistence, I configured the session cookie with `sameSite: 'none'` and `secure: true`. I also set `app.set('trust proxy', true)` in Express to trust Render's load balancer, and configured the Axios client on the frontend with `withCredentials: true` to ensure the browser sends cookies with cross-domain requests.

Q5: What is the purpose of `req.session.save()` in your routes, and what bug does it prevent?

> **Answer:** Express Session writes session data to the store asynchronously. If you call `res.redirect()` immediately after modifying `req.session`, the redirect response is sent to the browser before the write completes. The browser's next request will reach the backend before the session is saved, resulting in session loss (which causes a CSRF state mismatch error). Calling `req.session.save()` manually and redirecting inside its callback prevents this race condition.

Q6: How does the backend know whether the user wants to log into a Production/Developer org or a Sandbox?

> **Answer:** I added an environment selector dropdown on the frontend login page. When clicked, it passes a query parameter `?env=sandbox` or `?env=production` to the `/api/auth/login` endpoint. The backend reads this parameter, sets the login URL to `test.salesforce.com` or `login.salesforce.com`, and stores it in the session so that the callback and token exchange flows dynamically hit the correct Salesforce login gateway.

Q7: Why did you decide to ZIP the metadata package in memory instead of saving files to the server disk?

> **Answer:** Creating files on the server disk would lead to performance bottlenecks, disk I/O latency, and issues on platforms like Render or AWS where the filesystem is ephemeral (read-only or wiped on restart). It also presents file-locking issues if multiple users deploy simultaneously. Zipping in memory using standard streams is faster, cleaner, and has a zero-disk footprint.

Q8: How did you implement deploy polling on the frontend?

> **Answer:** When a user toggles a rule, the backend returns a `deployId` immediately. On the React dashboard, I set the rule card's state to "deploying" and call `pollDeploy(deployId, fullName)`. This function queries `/api/rules/deploy-status/:id` recursively every 2.5 seconds. Once the status returns `done: true`, I clear the card's deploying state, show a success toast, and refresh the list. I also use a `useRef` array to clean up active `setTimeout` timers on component unmount to prevent memory leaks.

Q9: How does the backend normalize the data? Why not return raw Salesforce JSON?

> **Answer:** Returning raw Salesforce API objects (with uppercase keys like `ValidationName` and `Active`) forces the frontend to couple tightly to Salesforce's schema. I implemented a normalization layer (`normalize`

`eRule()` in the backend route to transform records into standard camelCase JSON keys (`id`, `name`, `active`, `description`). This creates a clean API contract and makes it easy to swap Salesforce for a different CRM in the future without rebuilding the frontend.

Q10: How do you handle authentication expiration?

> **Answer:** I query the Salesforce User Info endpoint `/services/oauth2/userinfo` on session check. If the access token has expired (typically after 2 hours), the backend can use the `refresh_token` stored in the session to request a new access token from Salesforce automatically without forcing the user to log in again.

Part 6: Live Coding Modification Cheat Sheet

During the live interview, the interviewer may ask you to modify the project in front of them to test if you understand the codebase. Here are the 3 most common live tasks and exactly how to code them:

Scenario 1: "Can you change the app to show Contact or Opportunity validation rules instead of Account"

To change the target object from `Account` to `Contact` (or `Opportunity`), you only need to change the object name in **three places**:

1. In `server/services/sfTooling.js` (line 6):

* Locate the SOQL query in `getValidationRules()`:

```
- WHERE EntityDefinition.QualifiedApiName = 'Account'
+ WHERE EntityDefinition.QualifiedApiName = 'Contact'
```

2. In `server/routes/rules.js` (line 19):

* Locate the `normalizeRule()` function:

```
- objectName: record.EntityDefinition?.QualifiedApiName || 'Account',
+ objectName: record.EntityDefinition?.QualifiedApiName || 'Contact',
```

3. In `server/services/sfMetadata.js` (line 18):

* Locate the `deployRuleToggle()` function where the metadata `<fullName>` is compiled:

```
- <fullName>Account.${details.ValidationName}</fullName>
+ <fullName>Contact.${details.ValidationName}</fullName>
```

4. (Optional UI touch) In `client/src/pages/Dashboard.jsx` (line 119):

* Change the dashboard header text:

```
- <h2 className="dash-title">Account Validation Rules</h2>
+ <h2 className="dash-title">Contact Validation Rules</h2>
```

Scenario 2: "Can you add a real-time search bar to filter rules by name?"

This is a **pure frontend modification** in Dashboard.jsx. You can code this in 60 seconds:

1. Add a state variable for the search term (around line 20):

```
const [search, setSearch] = useState('')
```

2. Filter the rules array before rendering (around line 137):

```
const filteredRules = rules.filter(rule =>
  rule.name.toLowerCase().includes(search.toLowerCase())
)
```

3. Add the Search Input in the JSX (around line 123, right next to the Refresh button):

```
<input
  type="text"
  placeholder="Search rules..."
  value={search}
  onChange={(e) => setSearch(e.target.value)}
  style={{
    padding: '8px 12px',
    borderRadius: '4px',
    border: '1px solid var(--color-border)',
    fontSize: '14px',
    marginRight: '12px'
  }}
/>
```

4. Change the render map to use `filteredRules` instead of `rules` (around line 139):

```
- {rules.map(rule => (
+ {filteredRules.map(rule => (
```

Scenario 3: "Can you display the rule's validation Formula directly on the cards?"

This requires sending the formula from the backend and rendering it on the React card:

1. In the Backend Adapter: `server/routes/rules.js` (line 11):

* Standard Tooling API records do not include the formula in the main query list (they only have it in detailed metadata). But wait! We can modify the SOQL query in `server/services/sfTooling.js` to select the formula directly:

```
// In server/services/sfTooling.js:
- SELECT Id, ValidationName, Active, Description, ErrorMessage,
```

```
EntityDefinition.QualifiedApiName
+ SELECT Id, ValidationName, Active, Description, ErrorMessage, ErrorConditionFormula,
EntityDefinition.QualifiedApiName
```

* Then in the backend adapter `server/routes/rules.js` -> `normalizeRule()`:

```
// In server/routes/rules.js:
function normalizeRule(record) {
  return {
    ...
    description: record.Description || null,
    errorMessage: record.ErrorMessage || null,
+   formula: record.ErrorConditionFormula || null,
  };
}
```

2. In the Frontend Card Component: `client/src/components/RuleCard.jsx`:

* Open the file and render the formula inside a code tag:

```
// Add this inside the card markup in RuleCard.jsx:
{rule.formula && (
  <div className="rule-card-formula" style={{ marginTop: '10px' }}>
    <strong style={{ fontSize: '12px', color: 'var(--color-text-muted)' }}>Formula:</strong>
    <pre style={{
      background: '#f1f5f9',
      padding: '8px',
      borderRadius: '4px',
      fontSize: '11px',
      overflowX: 'auto',
      fontFamily: 'monospace'
    }}>{rule.formula}</pre>
  </div>
)}
```

Part 2: Line-by-Line Code Walkthrough

Salesforce Rule Manager -- Detailed Code Walkthrough

This document provides a line-by-line and section-by-section breakdown of the 10 core files in the project. It explains the design choices, what each block of code does, and why it is written that way.

1. `server/config/salesforce.js`

This file sanitizes your credentials and builds the authorize URL for Salesforce login.

```
const sanitizeEnvVar = (val) => {
  if (!val) return val;
  return val.replace(/["\\]/g, '').trim();
};
```

* **Explanation:** Accidental quotes (" , ') or backslashes (\) pasted into environment variables (like on Render's UI) will cause URL encoding errors or mismatch errors. This utility function uses regex to strip them out and trims any spaces or newlines.

```
const sfConfig = {
  clientId: sanitizeEnvVar(process.env.SF_CLIENT_ID),
  clientSecret: sanitizeEnvVar(process.env.SF_CLIENT_SECRET),
  callbackUrl: sanitizeEnvVar(process.env.SF_CALLBACK_URL) ||
'http://localhost:3001/api/auth/callback',
  loginUrl: sanitizeEnvVar(process.env.SF_LOGIN_URL) || 'https://login.salesforce.com',
  apiVersion: 'v59.0',
};
```

* **Explanation:** Creates a configuration object. If `SF_CALLBACK_URL` or `SF_LOGIN_URL` are missing in the environment, it falls back to standard developer defaults.

```
export function buildAuthorizeUrl(state, codeChallenge, loginUrl) {
  const params = new URLSearchParams({
    response_type: 'code',
    client_id: sfConfig.clientId,
    redirect_uri: sfConfig.callbackUrl,
    state: state || '',
    code_challenge: codeChallenge,
    code_challenge_method: 'S256',
  });
  const url = loginUrl || sfConfig.loginUrl;
```

```
return `${url}/services/oauth2/authorize?${params.toString()}`;
}
```

* **Explanation:** Generates the redirection URL. It accepts the PKCE `codeChallenge` and the dynamic `loginUrl` (sandbox vs production) and appends them to the OAuth authorization query.

2. `server/services/sfAuth.js`

Handles server-to-server OAuth requests directly to Salesforce.

```
export async function exchangeCodeForTokens(code, codeVerifier, loginUrl) {
  const url = loginUrl || sfConfig.loginUrl;
  const resp = await axios.post(`${url}/services/oauth2/token`, null, {
    params: {
      grant_type: 'authorization_code',
      code,
      client_id: sfConfig.clientId,
      client_secret: sfConfig.clientSecret,
      redirect_uri: sfConfig.callbackUrl,
      code_verifier: codeVerifier,
    },
  });
  const { access_token, refresh_token, instance_url, id } = resp.data;
  return { accessToken: access_token, refreshToken: refresh_token, instanceUrl: instance_url,
    idUrl: id };
}
```

* **Explanation:** This exchanges the temporary authorization code for real access and refresh tokens. It sends a POST request with the PKCE `code_verifier`. It returns the access token, the refresh token, the instance URL (the customer's Salesforce pod), and the identity URL.

```
export async function refreshAccessToken(refreshToken, loginUrl) {
  const url = loginUrl || sfConfig.loginUrl;
  const resp = await axios.post(`${url}/services/oauth2/token`, null, {
    params: {
      grant_type: 'refresh_token',
      refresh_token: refreshToken,
      client_id: sfConfig.clientId,
      client_secret: sfConfig.clientSecret,
    },
  });
  return {
    accessToken: resp.data.access_token,
    instanceUrl: resp.data.instance_url,
  };
}
```

* **Explanation:** If an access token expires (typically after 2 hours), this function sends a refresh token request to retrieve a new active access token silently, avoiding forcing the user to log in again.

3. `server/services/sfTooling.js`

Fetches rules from Salesforce using the Tooling API.

```
export async function getValidationRules(instanceUrl, accessToken) {
  const query = "SELECT Id, ValidationName, Active, Description, ErrorMessage,
EntityDefinition.QualifiedApiName FROM ValidationRule WHERE EntityDefinition.QualifiedApiName
= 'Account'";
  const resp = await axios.get(`${instanceUrl}/services/data/v59.0/tooling/query`, {
    headers: { Authorization: `Bearer ${accessToken}` },
    params: { q: query },
  });
  return resp.data.records;
}
```

* **Explanation:** Executes a SOQL query against the **ValidationRule** metadata table via the Tooling API. The query filters rules where the parent entity is **'Account'**. It passes the **accessToken** in the **Authorization** header.

```
export async function getValidationRuleDetails(instanceUrl, accessToken, ruleId) {
  const resp = await
  axios.get(`${instanceUrl}/services/data/v59.0/tooling/subjects/ValidationRule/${ruleId}`, {
    headers: { Authorization: `Bearer ${accessToken}` },
  });
  return resp.data;
}
```

* **Explanation:** Tooling API query lists don't include full XML metadata (like formulas or advanced parameters). This function queries the specific validation rule by its ID to get its complete detailed metadata (which is required before redeploying it).

4. `server/services/sfMetadata.js`

This is the core deployment engine. It builds XML files, compresses them in memory, and deploys them to the Metadata API.

```
const packageXml = `<?xml version="1.0" encoding="UTF-8"?>
<Package xmlns="http://soap.sforce.com/2006/04/metadata">
  <types>
    <members>Account</members>
    <name>CustomObject</name>
```

```

    </types>
    <version>59.0</version>
  </Package>`;

```

* **Explanation:** Standard Salesforce manifest file declaring that we are deploying changes to a **CustomObject** named **Account**.

```

function compileRuleXml(ruleDetails, isActive) {
  const metadata = ruleDetails.Metadata;
  const description = metadata.description ?
  `<description>${escapeXml(metadata.description)}</description>` : '';

  return `<?xml version="1.0" encoding="UTF-8"?>
  <CustomObject xmlns="http://soap.sforce.com/2006/04/metadata">
    <validationRules>
      <fullName>${ruleDetails.ValidationName}</fullName>
      <active>${isActive}</active>
      ${description}

    <errorConditionFormula>${escapeXml(metadata.errorConditionFormula)}</errorConditionFormula>
      <errorMessage>${escapeXml(metadata.errorMessage)}</errorMessage>
    </validationRules>
  </CustomObject>`;
}

```

* **Explanation:** Compiles the validation rule metadata back into Salesforce's native XML file layout, setting the **<active>** flag to **true** or **false** dynamically. We use **escapeXml** to escape symbols like **<**, **>**, or **&** inside formulas so they don't break the XML parser.

```

export async function deployRuleToggle(instanceUrl, accessToken, ruleDetails, isActive) {
  const archive = archiver('zip', { zlib: { level: 9 } });
  const zipChunks = [];

  archive.on('data', chunk => zipChunks.push(chunk));

```

* **Explanation:** Initializes the **archiver** zip engine with maximum compression (level 9). It creates an array **zipChunks** to capture the output stream as chunks in memory, avoiding saving files to the hard drive.

```

  archive.append(packageXml, { name: 'package.xml' });
  archive.append(ruleXml, { name: `objects/Account.object` });
  await archive.finalize();

  const zipBuffer = Buffer.concat(zipChunks);

```

* **Explanation:** Appends our **package.xml** and our dynamic **objects/Account.object** XML file containing the toggled validation rule to the ZIP. **archive.finalize()** tells the archiver we are done, and **Buffer.concat** merges the array of buffers into a single binary block.

```

const formData = new FormData();

```

```

    formData.append('file', zipBuffer, { filename: 'package.zip', contentType:
'application/zip' });

    const resp = await axios.post(`${instanceUrl}/services/data/v59.0/metadata/deploy`,
formData, {
    headers: {
      Authorization: `Bearer ${accessToken}`,
      ...formData.getHeaders(),
    },
  });
  return resp.data;
}

```

* **Explanation:** Creates a multipart form-data payload containing the binary ZIP buffer and sends it to the Metadata REST deployment endpoint. It returns a `deployId` which we can poll.

5. `server/routes/auth.js`

Handles login, logout, and token session persistence.

```

router.get('/login', (req, res) => {
  const env = req.query.env;
  const loginUrl = env === 'sandbox' ? 'https://test.salesforce.com' :
'https://login.salesforce.com';
  const state = crypto.randomBytes(16).toString('hex');

```

* **Explanation:** Reads the sandbox parameter. Generates a random `state` parameter to prevent CSRF attacks.

```

const codeVerifier = crypto.randomBytes(32).toString('base64url');
const codeChallenge = crypto
  .createHash('sha256')
  .update(codeVerifier)
  .digest('base64url');

```

* **Explanation:** Implements PKCE. Generates a random cryptographically secure string (the verifier), and hashes it using SHA256 (the challenge) encoded in URL-safe base64.

```

req.session.oauthState = state;
req.session.codeVerifier = codeVerifier;
req.session.loginUrl = loginUrl;

req.session.save((err) => {
  ...
  res.redirect(buildAuthorizeUrl(state, codeChallenge, loginUrl));
});
});

```

* **Explanation:** Saves variables to the session store. `req.session.save()` is called manually to ensure the session is completely written to the store *before* the redirect headers are sent to the browser, avoiding a session write race condition.

6. `server/routes/rules.js`

Handles listing, toggling, and checking status.

```
function normalizeRule(record) {
  return {
    id: record.Id,
    name: record.ValidationName,
    fullName: record.ValidationName,
    active: record.Active,
    description: record.Description || null,
    errorMessage: record.ErrorMessage || null,
    objectName: record.EntityDefinition?.QualifiedApiName || 'Account',
  };
}
```

* **Explanation:** This is an adapter. Salesforce returned fields have uppercase keys. The adapter normalizes them to camelCase properties so the React frontend is decoupled from the exact database column names of the Salesforce Tooling API.

```
router.put('/:fullName/toggle', async (req, res) => {
  const { accessToken, instanceUrl } = req.session.sf;
  const { fullName } = req.params;
  try {
    const records = await getValidationRules(instanceUrl, accessToken);
    const rule = records.find((r) => r.ValidationName === fullName);
    ...
    const details = await getValidationRuleDetails(instanceUrl, accessToken, rule.Id);
    const newState = !details.Metadata?.active;

    const deployResult = await deployRuleToggle(instanceUrl, accessToken, details, newState);
    res.json({
      message: `Deploy started`,
      deployId: deployResult.id,
    });
  }
  ...
})
```

* **Explanation:** Toggles a validation rule. It fetches rules, finds the requested rule, queries its detailed formula/metadata, flips the active boolean flag, calls the metadata ZIP deploy function, and returns the `deployId`.

7. `client/src/services/api.js`

Communicates with the backend and handles cross-domain hostname resolution.

```
const isLocal = window.location.hostname === 'localhost' || window.location.hostname ===
'127.0.0.1'
const apiBase = isLocal ? 'http://localhost:3001' :
'https://sf-rule-manager-backend.onrender.com'

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || apiBase,
  withCredentials: true,
})
```

* **Explanation:** Dynamically checks the browser hostname. If running on Vercel, it routes all API requests to the Render backend domain. If running locally, it defaults to port 3001. `withCredentials: true` tells the browser to include session cookies in cross-origin HTTP requests.

8. `client/src/hooks/useAuth.js`

A custom hook that coordinates user sessions.

```
export default function useAuth() {
  const [user, setUser] = useState(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    let cancelled = false
    async function checkSession() {
      try {
        const data = await getMe()
        if (!cancelled) setUser(data.user)
      } catch (err) {
        ...
      } finally {
        if (!cancelled) setLoading(false)
      }
    }
    checkSession()
    return () => { cancelled = true }
  }, [])
```

* **Explanation:** Checks the session on component mount. It calls `/api/auth/me` to see if the user is authenticated. If the response returns the user object, it sets the `user` state. `cancelled = true` on unmount prevents memory leak state updates if the page is changed mid-request.

9. `client/src/pages/Login.jsx`

The entry landing page component.

```
export default function Login() {
  const { isLoggedIn, loading, login } = useAuth()
  const [env, setEnv] = useState('production')
  const [searchParams, setSearchParams] = useSearchParams()
  const errorMsg = searchParams.get('error')
```

* **Explanation:** Reads current authentication state. Defines state for the dynamic login environment (**production** vs **sandbox**). Checks for an **error** query param in the redirect URL to capture callback failures.

```
  return (
    ...
    {errorMsg && (
      <ErrorBanner
        message={decodeURIComponent(errorMsg)}
        onDismiss={() => setSearchParams({}, { replace: true })}
      />
    )}
    ...
    <select value={env} onChange={(e) => setEnv(e.target.value)}>
      <option value="production">Production / Developer Org</option>
      <option value="sandbox">Sandbox</option>
    </select>
    ...
    <button className="login-btn" onClick={() => login(env)}>
      Login with Salesforce
    </button>
  )
}
```

* **Explanation:** Renders the login card. It displays the **ErrorBanner** if an error occurs, renders the environment selector dropdown, and triggers **login(env)** on click to pass the selection to the backend redirect.

10. `client/src/pages/Dashboard.jsx`

Main page managing grid loads, toggle states, and recursive polling.

```
async function pollDeploy(deployId, fullName) {
  try {
    const status = await getDeployStatus(deployId)
    if (status.done) {
      setDeployingRules(prev => {
        const next = { ...prev }
        delete next[fullName]
```

```

        return next
      })
      if (status.success) {
        showToast(`Rule "${fullName}" updated successfully`)
        fetchRules()
      } else {
        showToast(status.error || 'Deployment failed', 'error')
        fetchRules()
      }
      return
    }
    pollTimers.current[fullName] = setTimeout(() => pollDeploy(deployId, fullName), 2500)
  }
  ...

```

* **Explanation:** Recursively polls deploy status. If the status returns `done = false`, it registers a new timeout check in **2.5 seconds**. Once `done = true`, it cleans up the card's deploying state, triggers a data reload, and shows a toast notification.

```

useEffect(() => {
  return () => {
    Object.values(pollTimers.current).forEach(clearTimeout)
  }
}, [])

```

* **Explanation:** Essential cleanup function. When the user navigates away or logs out, this hook clears all active `setTimeout` timers to prevent background memory leaks.

11. `server/server.js`

The main entry file for spinning up the Node/Express backend.

```

import 'dotenv/config';
import app from './app.js';

const PORT = process.env.PORT || 3001;

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

* **Explanation:** Initializes the server. By importing `dotenv/config` first, we guarantee that all environment variables defined in the `.env` file (like client secrets, callback URLs, session secrets) are parsed and accessible via `process.env` before any routes or services are initialized. It then imports the Express application from `app.js` and starts listening on the designated `PORT`.

12. `server/app.js`

Configures the Express middleware, CORS policies, secure sessions, and API route mappings.

```
app.set('trust proxy', true);
```

* **Explanation:** Tells Express to trust incoming proxy headers (such as **X-Forwarded-Proto** and **X-Forwarded-For**). Since the backend is hosted on Render behind a load balancer/proxy, this is required so Express can correctly detect HTTPS requests and serve secure session cookies.

```
app.use(cors({
  origin: process.env.CLIENT_URL || 'http://localhost:5173',
  credentials: true,
}));
```

* **Explanation:** Configures Cross-Origin Resource Sharing. **origin** points to the React client's domain. Setting **credentials: true** enables the client browser to include the session cookie in cross-origin XMLHttpRequests/fetch calls to the backend API.

```
app.use(session({
  secret: process.env.SESSION_SECRET || 'dev-secret-change-in-prod',
  resave: false,
  saveUninitialized: false,
  cookie: {
    secure: isProd,
    sameSite: isProd ? 'none' : 'lax',
    httpOnly: true,
    maxAge: 1000 * 60 * 60 * 2,
  },
}));
```

* **Explanation:** Configures the session cookie. **resave: false** avoids resaving sessions that haven't changed. **saveUninitialized: false** avoids saving empty sessions, reducing store bloat. In production (**is Prod**), **secure** is set to **true** (forces HTTPS cookies) and **sameSite** is set to **'none'** (required for cross-domain cookie storage, since the React client is on Vercel and Express server is on Render).

13. `server/middleware/authCheck.js`

Protects backend API endpoints from unauthenticated access.

```
export default function authCheck(req, res, next) {
  if (!req.session?.sf) {
    return res.status(401).json({ error: 'Not authenticated -- please log in first' });
  }
  next();
}
```

* **Explanation:** Intercepts requests destined for protected routes (e.g., retrieving or toggling validation rules). If `req.session.sf` (which holds the access token and login credentials) is missing or undefined, it short-circuits the request chain and returns an HTTP `401 Unauthorized` response. Otherwise, it calls `next()` to pass control to the route handler.

14. `client/src/main.jsx`

The client-side entry point that bootstraps React.

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

* **Explanation:** Creates the React root under the `#root` div in the HTML structure. It wraps the app inside `BrowserRouter` to activate React Router functionality, and inside React's `StrictMode` to identify potential warnings and rendering lifecycle issues during development.

15. `client/src/App.jsx`

Declares page-level routing.

```
export default function App() {
  return (
    <Routes>
      <Route path="/" element={<Login />} />
      <Route path="/dashboard" element={<Dashboard />} />
      <Route path="*" element={<NotFound />} />
    </Routes>
  )
}
```

* **Explanation:** Defines the routing structure using React Router. The home path `/` points to the `Login` screen, `/dashboard` displays the dashboard, and `*` is a wildcard path rendering the `NotFound` page when a user hits an unrecognized URL.

16. `client/src/components/RuleCard.jsx`

A reusable card component representing a single Salesforce Validation Rule.

```
export default function RuleCard({ rule, onToggle, isDeploying }) {
  const isActive = rule.active

  return (
    <div className={`rule-card ${isDeploying ? 'rule-card--deploying' : ''}`}>
      ...
      <span className={`rule-badge ${isActive ? 'rule-badge--active' : 'rule-badge--inactive'}`}>
        {isActive ? 'Active' : 'Inactive'}
      </span>
      ...
      {isDeploying ? (
        <span className="rule-card-deploying">Deploying?</span>
      ) : (
        <label className="toggle">
          <input
            type="checkbox"
            checked={isActive}
            onChange={() => onToggle(rule.fullName)}
            disabled={isDeploying}
          />
          <span className="toggle-slider" />
        </label>
      )}
    </div>
  )
}
```

* **Explanation:** Handles styling and interactive toggle input for a rule. Clicking the toggle triggers `onToggle` with the rule's `fullName`. While the deploy is in progress (`isDeploying = true`), the toggle is replaced with a "Deploying..." status tag and the card gets dimmed using CSS classes.

17. `client/src/components/Header.jsx`

The application's persistent header bar.

```
export default function Header({ user, onLogout }) {
  return (
    <header className="header">
      ...
      {user && (
        <div className="header-right">
          <div className="header-user">
            <span className="header-username">{user.name}</span>
            {user.orgName && <span className="header-org">{user.orgName}</span>}
          </div>
          <button className="header-logout" onClick={onLogout}>Log out</button>
        </div>
      )}
    </header>
  )
}
```

```

    </div>
  )}
</header>
)
}

```

* **Explanation:** Provides navigational structure. Renders the logo brand and the user's name and organization (retrieved dynamically from the Salesforce session) next to a "Log out" button.

18. `client/src/components/ErrorBanner.jsx`

Displays error alerts to the user.

```

export default function ErrorBanner({ message, onDismiss }) {
  if (!message) return null

  return (
    <div className="error-banner" role="alert">
      <p className="error-banner-msg">{message}</p>
      {onDismiss && (
        <button className="error-banner-dismiss" onClick={onDismiss}
aria-label="Dismiss">?</button>
      )}
    </div>
  )
}

```

* **Explanation:** A dismissible component used mainly on the Login screen to alert users when a connection or authorization failure happens. It extracts and displays the URI-decoded **error** query parameter.

19. `client/src/components/Loader.jsx`

A lightweight, theme-friendly CSS loading spinner.

```

export default function Loader({ size = 28, className = '' }) {
  return (
    <>
      <style>{@keyframes spin { to { transform: rotate(360deg) } }}</style>
      <div
        className={className}
        style={{ ...spinnerStyle, width: size, height: size }}
        role="status"
        aria-label="Loading"
      />
    </>
  )
}

```

```
)  
}
```

* **Explanation:** Renders a customizable circular spinner using standard keyframe animations. Placing the `<style>` tag dynamically inside the React render makes it fully self-contained and easy to reuse without requiring external styles.